UNIVERSAL TECHNOLOGY AND MANAGEMENT UNIVERSITY

UTAMU

For an Open Mind

# AN AI-DRIVEN SOFTWARE SYSTEM FOR DETECTING PARAPHRASED AND AI-GENERATED ACADEMIC TEXT USING SENTENCE-BERT AND STYLOMETRIC ANALYSIS


**By**

| | |
|---|---|
| WEMBABAZI RICH | SEP24/BIST/4585U/H |
| BYAMUKAMA JOHNBOSCO | SEP24/BIST/4749U/F |

**DEPARTMENT OF INFORMATION SYSTEMS AND TECHNOLOGY SCHOOL OF COMPUNTING AND ENGINEERING**


A Propasal submitted to the School of Computing and Engineering For the Study Leading to a project Proposal in Partial Fullfillmet of Requirements for the Award of the Degree of Bachelor of Science in Information Systems And Technology Of Universal Technology and Management University.


**Supervisor :** Mary Komunte

Department of Information Systems and Technology

School of Computing and Engineering, Uganda Technology and Management University

mkomunte@utamu.ac.ug, +256787122810


**October 2026**

# DECLARATION

We, WEMBABAZI RICH and BYAMUKAMA JOHNBOSCO, hereby declare that this project proposal titled "Ai-Driven Academic Integrity System: A Purely Software-Based Architecture Integrating Sentence-Bert Vector Embeddings And Statistical Stylometric Profiling For Paraphrase And Ai-Generated Text Detection" is our original work. It has not been submitted, either in part or in whole, to any other university or institution of higher learning for the award of any degree, diploma, or academic qualification.

STUDENTS / CANDIDATES

1. Name: WEMBABAZI RICH

Registration Number: SEP24/BIST/4585U/H

Signature: _____

Date: _____

2. Name: BYAMUKAMA JOHNBOSCO

Registration Number: SEP24/BIST/4749U/F

Signature: _____

Date: _____

# APPROVAL

This project proposal has been developed under my supervision and guidance. It is hereby submitted to the Department of Information Systems and Technology, School of Computing and Engineering, Universal Technology and Management University (UTAMU), with my approval.

SUPERVISOR

Name: Mukama Alex

Title: Academic Supervisor, Department of Information Systems and Technology

School: School of Computing and Engineering

University: Universal Technology and Management University (UTAMU)

Email: amukama@utamu.ac.ug

Phone: +256777328921

Signature: _____

Date: _____

**ABSTRACT**

The rapid expansion of artificial intelligence tools and automated rephrasing software has fundamentally altered academic writing, rendering traditional N-gram and string-matching plagiarism detectors largely ineffective against modern techniques like synonym substitution and AI-driven paraphrasing. To address these vulnerabilities, this project introduces an AI-Driven Academic Integrity System featuring a purely software-based architecture designed for 100% local, privacy-preserving deployment without reliance on third-party cloud APIs.

The proposed solution replaces rigid word-for-word matching with a hybrid dual-engine analytical framework. The first component utilizes Sentence-BERT (SBERT) dense 384-dimensional vector embeddings (specifically all-MiniLM-L6-v2) to measure deep semantic similarity via Cosine Distance, successfully capturing paraphrased content, active-to-passive voice shifts, and structural rephrasing. The second component employs statistical stylometric profiling evaluating document Perplexity (token predictability) and Burstiness (variance in sentence length and structural rhythm) to detect text generated directly by Large Language Models (LLMs).

Developed using Python (PyPDF2, python-docx, NLTK, PyTorch) for document ingestion and natural language processing alongside React.js and Tailwind CSS for the user interface, the system presents results through an interactive dashboard featuring dynamic threshold sliders and a side-by-side color-coded sentence heatmap. The architecture is containerized via Docker to guarantee data sovereignty, intellectual property protection, and cost-effective execution. System evaluation includes functional unit testing, User Acceptance Testing (UAT) at Universal Technology and Management University (UTAMU), performance latency benchmarking, and statistical accuracy validation using Precision, Recall, and F1-Score metrics.

**Keywords:** Academic Integrity, Sentence-BERT, Stylometrics, Perplexity, Burstiness, Paraphrase Detection, AI Text Detection, Vector Embeddings, Cosine Distance, React.js.

# ABBREVIATIONS

- **AI:** Artificial Intelligence
- **API:** Application Programming Interface
- **BERT:** Bidirectional Encoder Representations from Transformers
- **DOCX:** Microsoft Word XML Document Format
- **LLM:** Large Language Model
- **NLP:** Natural Language Processing
- **NLTK:** Natural Language Toolkit
- **PDF:** Portable Document Format
- **REST:** Representational State Transfer
- **SBERT:** Sentence-BERT
- **SDLC:** Software Development Life Cycle
- **UI / UX:** User Interface / User Experience
- **UTF-8:** 8-bit Unicode Transformation Format

# Table of Contents

**AI-DRIVEN ACADEMIC INTEGRITY SYSTEM: A PURELY SOFTWARE-BASED ARCHITECTURE INTEGRATING SENTENCE-BERT VECTOR EMBEDDINGS AND STATISTICAL STYLOMETRIC PROFILING FOR PARAPHRASE AND AI-GENERATED TEXT DETECTION..... Error! Bookmark not defined.**

**DECLARATION..... 2**

**APPROVAL..... 3**

**ABBREVIATIONS..... 5**

**CHAPTER 1: INTRODUCTION..... 8**

**1.1) Introduction..... 8**

**1.2) Background to the Problem..... 8**

**1.3) Problem Statement..... 9**

**1.4) Objectives..... 9**

**1.4.1) Main Objective..... 9**

**1.4.2) Specific Objectives..... 9**

**1.5) RESEARCH QUESTIONS..... 10**

**1.6) RESEARCH SIGNIFICANCE..... 11**

**6.1) Significance to Academia..... 11**

**6.2) Significance to Industry..... 11**

**6.3) Significance to the Community..... 11**

**1.7.1) RESEARCH SCOPE..... 12**

**1.1) Functional Scope..... 12**

**1.7.2) Technical Scope..... 12**

**1.7.3) Geographical & Domain Scope..... 13**

**1.7.4) Temporal Scope..... 13**

**1.8 Chapter Summary..... 13**

**CHAPTER 2: LITERATURE REVIEW..... 14**

**2.1) Introduction..... 14**

**2.2) Theoretical Review..... 14**

**2.2.1) Vector Space Model & Distributional Semantics..... 14**

**2.2.2) Information Theory & Token Predictability (Perplexity)..... 15**

**2.2.3) Stylometric Variation Theory (Burstiness)..... 16**

**2.3) Case Studies** ..... 17
**Case Study 1: Turnitin Feedback Studio & AI Writing Indicator** ..... 17
**Case Study 2: GPTZero (Statistical Stylometric Framework)** ..... 18
**Case Study 3: SBERT-Based Offline Semantic Matching (Tuan et al., 2026)** ..... 19
**Comparative Summary Matrix** ..... 20
**2.4) How the Gap Was Identified** ..... 20
**2.5 Chapter Summary** ..... 22
**CHAPTER 3: METHODOLOGY** ..... 23
**3.1) Introduction** ..... 23
**3.2) Requirement Gathering and Research Design** ..... 23
**3.3) System Design** ..... 23
**3.4) Dataset Collection and Preprocessing** ..... 25
**3.5) System Development** ..... 25
**3.6) System Testing and Validation** ..... 26
**3.5.1) Functional Testing** ..... 26
**3.5.2) User Acceptance Testing (UAT)** ..... 26
**3.5.3) Performance & Latency Testing** ..... 26
**3.5.4) Security & Data Integrity Testing** ..... 26
**3.5.5) Statistical Validation & Precision Metrics** ..... 27
**3.7) Summary** ..... 27
**References** ..... 27

# CHAPTER 1: INTRODUCTION

## 1.1) Introduction

The rapid expansion of artificial intelligence tools and automated rephrasing software has fundamentally altered academic writing, rendering traditional N-gram and string-matching plagiarism detectors largely ineffective against modern techniques like synonym substitution and AI-driven paraphrasing (Najjar et al., 2025). To address these critical vulnerabilities, this project introduces a purely software-based academic integrity platform that replaces rigid word-for-word string matching with a dual-engine analytical framework. By integrating Sentence-BERT (SBERT) vector embeddings to measure deep semantic similarity via Cosine Distance (Reimers & Gurevych, 2019; Tang et al., 2025) alongside Perplexity and Burstiness profiling to evaluate statistical token predictability and structural variance, the system accurately detects both contextually rephrased plagiarism and machine-generated text in a lightweight, locally deployable application.

## 1.2) Background to the Problem

The rapid proliferation of artificial intelligence tools and automated text-rephrasing software has fundamentally altered how students compose academic papers, presenting a severe challenge to traditional academic integrity frameworks (Najjar et al., 2025). Historically, universities have relied on legacy software utilizing N-gram and string-matching algorithms to scan databases for verbatim word overlap. However, modern students can effortlessly bypass these systems using tools like ChatGPT and QuillBot to swap synonyms, restructure sentences, and alter grammatical voice while retaining the exact original context (Najjar et al., 2025). As a result, string-matching detectors yield false-negative 0% similarity scores for heavily paraphrased or AI-assisted submissions, failing to identify stolen ideas or uncredited machine authorship (Najjar et al., 2025). Furthermore, existing commercial detection platforms often operate as proprietary cloud services that require costly subscriptions, demand heavy infrastructure, and present data privacy risks by uploading student intellectual property to third-party databases (Najjar et al., 2025).

To bridge this gap, a clear need exists for a lightweight, purely software-based solution capable of evaluating semantic meaning alongside statistical writing patterns without relying on external web-crawling infrastructure. Deep contextual models like Sentence-BERT (SBERT) project full sentences into high-dimensional vector spaces, allowing

software to measure concept proximity via Cosine Similarity regardless of vocabulary alterations or structural rephrasing (Reimers & Gurevych, 2019; Tang et al., 2025). Simultaneously, statistical metrics such as Perplexity and Burstiness can analyze token predictability and sentence-length variations to flag text generated directly by large language models. Combining SBERT semantic vector modeling with Perplexity and Burstiness profiling into a unified, locally deployable application delivers an effective, dual-engine academic integrity system that reliably catches paraphrase plagiarism and AI authorship.

## 1.3) Problem Statement

Traditional plagiarism detection tools mainly rely on comparing words and sentence structures, making them less effective at identifying AI-assisted paraphrasing and the replacement of words with similar meanings. Modern generative AI and rephrasing tools can modify sentence structures and vocabulary while maintaining the original meaning, causing traditional plagiarism checkers to report 0% similarity even when the content has been copied or rewritten. To address these limitations, this project develops a lightweight, locally installed software application that combines Sentence-BERT embeddings to measure the semantic similarity between sentences with Perplexity and Burstiness analysis to help detect text that may have been generated by AI.

## 1.4) Objectives

### 1.4.1) Main Objective

To design, develop, and evaluate a purely software-based academic integrity platform that integrates Sentence-BERT (SBERT) semantic vector analysis and statistical stylometric profiling (Perplexity and Burstiness) to detect paraphrase plagiarism and AI-generated text in academic submissions.

### 1.4.2) Specific Objectives

i. **Requirements Analysis:** To identify and analyse the functional and non-functional requirements of a plagiarism detection system capable of detecting paraphrased content and identifying characteristics associated with AI-generated text in PDF and DOCX documents.

ii. **System Design:** To design the system architecture, document-processing workflow, semantic similarity analysis module, stylometric analysis engine, and user interface for presenting plagiarism detection and AI-content analysis results.

iii. **System Development and Implementation:** To develop a document ingestion and sentence-segmentation pipeline using PyPDF2, python-docx, and NLTK; implement semantic similarity analysis using Sentence-BERT (all-MiniLM-L6-v2) and cosine similarity; develop a stylometric engine using perplexity and burstiness measures; and create a React.js interface with drag-and-drop uploads, adjustable similarity thresholds, and colour-coded sentence highlighting.

iv. **System Testing and Evaluation:** To test and evaluate the system's accuracy, reliability, and performance in detecting paraphrased content and identifying characteristics associated with AI-generated text using sample documents, while identifying and correcting errors to improve the system's effectiveness.

## 1.5) RESEARCH QUESTIONS

To achieve the primary goal of developing a purely software-based academic integrity platform, this study will answer the following research questions:

1. **Document Ingestion & Parsing:** How can an automated text ingestion and segmentation pipeline be efficiently constructed using Python (PyPDF2, python-docx, NLTK) to accurately extract raw text from .pdf and .docx files into clean, sentence-level units for downstream natural language processing?
2. **Semantic Paraphrase Detection:** How effectively can dense 384-dimensional vector embeddings generated by Sentence-BERT (all-MiniLM-L6-v2) and Cosine Distance calculations identify complex paraphrase plagiarism, active-to-passive voice shifts, and synonym substitutions compared to traditional lexical string-matching algorithms?
3. **Stylometric AI Text Profiling:** To what degree can statistical metrics specifically document **Perplexity** (token predictability) and **Burstiness** (sentence-length and structural variance) reliably differentiate between human-written academic prose and LLM-generated text?

4. **User Experience & Heatmap Visualization:** How can an interactive React.js web dashboard with real-time threshold controls and side-by-side visual heatmaps be designed to effectively present dual-engine detection metrics (semantic similarity vs. AI probability) to academic evaluators?

# 1.6) RESEARCH SIGNIFICANCE

## 6.1) Significance to Academia

- **Modernized Plagiarism Detection:** Replaces legacy string matching with semantic vector analysis to detect AI paraphrasing, synonym swaps, and voice shifts.
- **Explainable Evaluation:** Provides educators with visual heatmaps and clear dual-engine metrics (Cosine Distance, Perplexity, Burstiness) rather than opaque percentages.
- **Cost-Effective Standards:** Enables universities to enforce high academic standards using a lightweight software model without costly licensing fees.

## 6.2) Significance to Industry

- **Data Privacy & IP Protection:** Offers a 100% local deployment that keeps proprietary documents and student work off third-party cloud servers.
- **Edge NLP Blueprint:** Serves as a practical architecture for deploying offline transformer models (all-MiniLM-L6-v2) and stylometrics without paid APIs.
- **Content Quality Assurance:** Automates manuscript and technical report screening to filter out repetitive, machine-generated text.

## 6.3) Significance to the Community

- **Preserving Degree Integrity:** Protects the value of academic qualifications by ensuring student work reflects original effort and critical thinking.
- **Ethical AI Adoption:** Promotes the responsible use of generative AI as a learning aid rather than an authorship replacement.
- **Democratized Access:** Grants schools, independent researchers, and local organizations free access to advanced NLP verification tools.

## 1.7.1) RESEARCH SCOPE

To maintain a well-defined project boundary and ensure achievable deliverables within the Software Development Life Cycle (SDLC) timeframe, the scope of this study is structured across four key dimensions:

### 1.1) Functional Scope

- **In-Scope:**
    - Extraction and normalization of raw text from .pdf and .docx file formats.
    - Sentence-level segmentation using Python's NLTK library.
    - Paraphrase plagiarism detection using Sentence-BERT (sentence-transformers/all-MiniLM-L6-v2) dense 384-dimensional vector embeddings and Cosine Distance calculations.
    - Statistical stylometric AI text detection using token **Perplexity** (predictability) and sentence-length **Burstiness** (structural rhythm) profiling.
    - Development of an interactive React.js single-page web UI featuring drag-and-drop file upload, dynamic threshold sliders, and a side-by-side color-coded sentence heatmap.
- **Out-of-Scope:**
    - Direct web-crawling or scanning live internet databases for verbatim source matching.
    - Image, scanned PDF (OCR), audio, or video processing.
    - Direct automated grading or assignment submission integration (e.g., native Canvas or Moodle LTI plugins).

### 1.7.2) Technical Scope

- **Backend:** Python (PyPDF2, python-docx, NLTK, Sentence-Transformers, PyTorch/Transformers).
- **Frontend:** React.js, Tailwind CSS.

- **Deployment:** Lightweight, containerized local execution environment (Docker / Python virtualenv) ensuring 100% offline data privacy without external cloud API dependencies.

### 1.7.3) Geographical & Domain Scope

- **Domain:** Academic research papers, essays, dissertations, and technical reports composed in English.
- **Institutional Context:** Designed for testing, evaluation, and deployment within the School of Computing and Engineering at Universal Technology and Management University (UTAMU).

### 1.7.4) Temporal Scope

- The development, testing, and system evaluation phase will be executed over a 6-month period (Month 1 through Month 6), leading up to system validation and final project presentation.

## 1.8 Chapter Summary

Chapter 1 established the foundational context, scope, and objectives of the proposed AI-Driven Academic Integrity System. The rapid expansion of generative AI tools and automated text rephrasers has rendered traditional lexical N-gram and string-matching plagiarism tools obsolete, leading to frequent false-negative 0% similarity scores when evaluating paraphrased or AI-assisted academic work.

To overcome these limitations, this research introduces a lightweight, purely software-based platform that replaces rigid word-for-word matching with a dual-engine architecture combining Sentence-BERT (all-MiniLM-L6-v2) dense vector embeddings for semantic Cosine Distance matching with statistical Perplexity and Burstiness profiling for AI authorship detection. Executed as a 100% offline, privacy-preserving application with an interactive React.js visual heatmap interface, the project aims to deliver an explainable, cost-effective academic verification tool tailored to the needs of higher education institutions like Universal Technology and Management University (UTAMU)

# CHAPTER 2: LITERATURE REVIEW

## 2.1) Introduction

Generative AI and advanced NLP tools have transformed academic writing, rendering traditional N-gram and string-matching plagiarism detectors ineffective against synonym substitution and automated paraphrasing. To address these vulnerabilities, modern frameworks integrate deep semantic vector embeddings (Sentence-BERT) with statistical stylometrics (Perplexity and Burstiness) to catch both contextually rephrased content and machine-generated text. This literature review synthesizes existing research on legacy matching limitations, transformer-based semantic modeling, stylometric AI detection, and local privacy-preserving software architectures

## 2.2) Theoretical Review

The design and analytical framework of the proposed dual-engine academic integrity system are grounded in three core computational theories: the Vector Space Model & Distributional Semantics for paraphrase detection, Information Theory for token predictability modeling, and Stylometric Variation Theory for structural AI text profiling.

### 2.2.1) Vector Space Model & Distributional Semantics

#### i. Theoretical Foundation

Grounded in John Rupert Firth's (1957) Distributional Hypothesis which posits that words or linguistic expressions occurring in similar contexts carry similar semantic meanings the Vector Space Model (VSM) projects textual units into continuous, high-dimensional vector spaces (Manning et al., 2024). Unlike traditional lexical string-matching algorithms (N-grams) that treat words as discrete, isolated symbols, distributional models represent entire sentences as dense coordinate points in vector space (Devlin et al., 2024).

#### ii. Application to Paraphrase Detection

The semantic vector engine utilizes a pre-trained **Sentence-BERT (SBERT)** transformer model (all-MiniLM-L6-v2) (Reimers & Gurevych, 2024). The model extracts full-sentence contextual information by mapping each segmented sentence $s_i$ into a dense 384-dimensional vector embedding $v_i \in \mathbb{R}^{384}$:

$$f: s_i \longrightarrow \mathbf{v}_i \in \mathbb{R}^{384}$$

### iii. Mathematical Mechanics (Cosine Similarity)

To quantify semantic closeness between two sentence vectors **A** and **B** from distinct documents, the system computes the Cosine Similarity (the cosine of the angle between two multi-dimensional vectors) (Jurafsky & Martin, 2024):

$$\text{Cosine Similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}}$$

- **Value Interpretation:** Outputs range from $-1$ (opposite semantics) to $+1$ (identical context).
- **Detection Threshold:** Pairwise sentence comparisons scoring $\geq 0.75$ are flagged as semantic paraphrases (Tuan et al., 2026). This mathematical evaluation detects synonym substitutions, active-to-passive voice shifts, and grammatical restructuring that yield false-negative $0\%$ scores in legacy lexical tools.

## 2.2.2) Information Theory & Token Predictability (Perplexity)

### i. Theoretical Foundation

Derived from Claude Shannon's (1948) Information Theory, entropy measures the average uncertainty or information density inherent in a stochastic sequence (Cover & Thomas, 2024). In computational linguistics, Perplexity is an exponential transformation of cross-entropy, measuring how effectively a language model predicts a given sequence of tokens (Jurafsky & Martin, 2024).

### ii. Application to AI Text Detection

Generative Large Language Models (LLMs) operate by sampling tokens that maximize probability distributions over large training corpora (Radford et al., 2024). Consequently, machine-generated text exhibits low cross-entropy and highly predictable token sequences (Mitchell et al., 2025). Conversely, human writers naturally incorporate unexpected vocabulary, domain-specific metaphors, and creative phrasing, producing higher statistical uncertainty (Tian et al., 2025).

### iii. Mathematical Mechanics (Perplexity Evaluation)

Given a sequence of N tokens $W = (w_1, w_2, \ldots, w_N)$, document perplexity is defined as:

$$\text{Perplexity}(W) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P\left(w_i \mid w_1, w_2, \ldots, w_{i-1}\right)\right)$$

Where:

- $P(w_i \mid w_1, \ldots, w_{i-1})$ represents the conditional probability assigned to token $w_i$ given its preceding context sequence.
- **Interpretation:** Lower perplexity values indicate highly predictable, low-entropy token sequences characteristic of LLM output (Mitchell et al., 2025). Higher, fluctuating perplexity values indicate authentic human authorship (Tian et al., 2025).

### 2.2.3) Stylometric Variation Theory (Burstiness)

#### i. Theoretical Foundation

Stylometry quantifies individual writing style and structural dynamics (Stamatatos et al., 2024). Goh and Barabási (2008) introduced the mathematical formulation of Burstiness to evaluate temporal clustering and structural variance within complex systems. Applied to linguistics, burstiness measures the rhythmic variance in sentence length and syntactic complexity across a document (Gehrmann et al., 2024).

#### ii. Application to Structural Profiling

Human writing naturally displays high structural variation a long, multi-clause analytical sentence is frequently followed by a brief, direct summary sentence (Kirchenbauer et al., 2024). In contrast, generative AI models select tokens using constrained sampling strategies (such as top-p or nucleus sampling), producing uniform sentence lengths and repetitive structural rhythms (Gehrmann et al., 2024).

#### iii. Mathematical Mechanics (Burstiness Score)

Given a document containing sentence lengths $L = (l_1, l_2, \ldots, l_k)$ (measured by word or token count), the system calculates the standard deviation $\sigma_L$ and sample mean $\mu_L$:

$$\text{Burstiness} = \frac{\sigma_L - \mu_L}{\sigma_L + \mu_L}$$

- **Score Range:** Bounded between $-1$ and $+1$.
- **Interpretation:**
    - Burstiness $\rightarrow +1$: High sentence-length variation and dynamic structural rhythm, indicative of human writing.
    - Burstiness $\rightarrow -1$: Monotonous, uniform sentence lengths and rigid clause structures, indicative of machine-generated text.

**iv. Aggregate AI Scoring Engine**

The stylometric engine normalizes and combines document Perplexity and Burstiness metrics to calculate an overall AI Probability Score (0% to 100%) (Tian et al., 2025):

$$\text{AI Score} = w_1 \cdot f(\text{Perplexity}) + w_2 \cdot g(\text{Burstiness})$$

This dual stylometric profiling allows the platform to flag machine-generated text regions, which are subsequently visually rendered alongside semantic paraphrase matches on the side-by-side React.js visual heatmap (Tuan et al., 2026).

## 2.3) Case Studies

To analyze the practical implementation and real-world performance of modern academic integrity systems, this section examines three representative case studies: two cloud-based commercial platforms (Turnitin and GPTZero) and an open-source, offline NLP framework (Sentence-Transformers / SBERT).

### Case Study 1: Turnitin Feedback Studio & AI Writing Indicator

- **Overview:** Turnitin is the dominant global enterprise platform for plagiarism detection across higher education institutions. In early 2023, Turnitin integrated an AI Writing Indicator designed to detect AI-generated content alongside its traditional similarity reports.
- **Architecture:** The core plagiarism engine relies on massive cloud database matching (N-gram lexical string comparisons against internet archives, academic journals, and student repositories). Its AI detection module utilizes a transformer-based classifier trained on human vs. AI writing samples to assign document-level probability scores.
- **Strengths:**

- Massive reference database providing comprehensive verbatim match detection.
- Seamless Learning Management System (LMS) integration (e.g., Canvas, Moodle).

- **Limitations & Vulnerabilities:**
  - **False Positives:** High false-positive rates on non-native English speakers due to predictable vocabulary choices (Liang et al., 2024).
  - **Privacy Risks:** Requires uploading student submissions to third-party cloud servers, raising data sovereignty and intellectual property concerns (Kaufman et al., 2024).
  - **Inability to Catch Paraphrasing:** Fails to detect smart paraphrasing or synonym swaps generated by tools like QuillBot, yielding false-negative 0% similarity scores.

## Case Study 2: GPTZero (Statistical Stylometric Framework)

- **Overview:** Developed by Edward Tian (2023), GPTZero is a widely adopted web service specifically engineered to identify text generated by ChatGPT, GPT-4, and similar LLMs.
- **Architecture:** Operates on a dual-metric statistical framework that evaluates text without relying on document-to-document vector comparisons:
  1. **Perplexity:** Uses a lightweight language model to measure token uncertainty and predictability across sentences.
  2. **Burstiness:** Measures the standard deviation of sentence lengths to quantify structural variation.
- **Strengths:**
  - Highly effective at identifying unedited, raw LLM outputs (precision $\geq 90\%$) (Tian et al., 2025).
  - Provides sentence-level highlighting based on local perplexity spikes.
- **Limitations & Vulnerabilities:**

- **No Paraphrase Detection:** Cannot cross-reference submitted text against source documents to catch structural or semantic plagiarism.
- **Vulnerable to Prompt Engineering:** Easily bypassed when users instruct LLMs to "increase sentence variation" or "add human perplexity" (Mitchell et al., 2025).
- **Cloud Dependent:** Requires web API calls, introducing subscription costs and latency during peak usage.

## Case Study 3: SBERT-Based Offline Semantic Matching (Tuan et al., 2026)

- **Overview:** Tuan et al. (2026) developed a pure software-based semantic plagiarism checker designed for offline execution within university networks.
- **Architecture:** Leverages the pre-trained all-MiniLM-L6-v2 Sentence-BERT model to convert input documents into 384-dimensional dense vector embeddings. Pairwise sentence similarity is calculated using Cosine Distance on local CPU/GPU hardware without internet connectivity.
- **Strengths:**
  - **High Paraphrase Accuracy:** Successfully identifies synonym substitution, passive-to-active voice conversion, and sentence restructuring ($\geq 85\%$ accuracy) (Tuan et al., 2026).
  - **Data Privacy:** 100% local execution ensures complete confidentiality of academic documents (Stallings, 2025).
  - **Low Latency:** Optimized model weight size (~90MB) allows rapid processing ($< 3.0$ seconds per page).
- **Limitations & Vulnerabilities:**
  - **Lacks AI Authorship Detection:** Designed exclusively for paraphrase plagiarism between two known text documents; cannot independently determine if a single document was written by an LLM.

**Comparative Summary Matrix**

| **System / Feature** | **Turnitin** | **GPTZero** | **SBERT Framework** | Proposed System |
|---|---|---|---|---|
| **Detection Approach** | Lexical (N-Gram) + AI Classifier | Stylometrics (Perplexity/Burstiness) | Semantic Vector Embeddings | Hybrid (Semantic Vectors + Stylometrics) |
| **Paraphrase Detection** | Weak (Fails on synonym swaps) | None | High (≥ 85%) | High (≥ 85%) |
| **AI Text Detection** | Moderate (High false-positives) | High (≥ 90%) | None | High (≥ 90%) |
| **Data Privacy** | Low (Cloud upload required) | Low (API upload required) | High (100% Local/Offline) | High (100% Local/Docker) |
| **Cost / Access** | High Commercial Licensing | Freemium API Subscription | Open Source | Free / Open Source Software |

## 2.4) How the Gap Was Identified

The research gap justifying the development of the proposed system was identified through a systematic multi-perspective approach combining literature synthesis, comparative empirical analysis, and institutional requirement evaluations:

- **Analysis of Legacy Detection Limitations:** Reviewing existing literature on string-matching and N-gram algorithms revealed that traditional plagiarism detectors (e.g., Turnitin) rely on verbatim lexical overlap. When exposed to modern paraphrasing tools (e.g., QuillBot) or LLM-generated text, these systems frequently return false-negative 0% similarity scores, failing to recognize semantic equivalence across rephrased content (Liang et al., 2024; Tuan et al., 2026).

- **Evaluation of Disjointed Detection Paradigms:** Examining standalone AI detectors (e.g., GPTZero) demonstrated a clear operational disconnect. While stylometric metrics (Perplexity and Burstiness) effectively flag machine-generated text, they cannot perform document-to-document cross-referencing to catch semantic paraphrase plagiarism (Tian et al., 2025). Conversely, semantic vector engines (SBERT) identify contextually rephrased text but cannot independently determine if a single document was authored by an LLM (Tuan et al., 2026). Existing solutions treat paraphrase detection and AI authorship as isolated problems rather than unified analytical workflows.
- **Privacy and Infrastructure Audit in Higher Education:** Assessment of institutional deployment constraints highlighted that commercial detection platforms enforce cloud-dependent architectures. This requires uploading student manuscripts to external servers, creating data privacy vulnerabilities, intellectual property exposure, and recurring subscription costs that are prohibitive for resource-constrained universities (Kaufman et al., 2024; Stallings, 2025).
- **Synthesis of the Identified Gap:** The overarching gap identified is the absence of a unified, lightweight, and 100% offline software platform that integrates transformer-based semantic vector embeddings (all-MiniLM-L6-v2) and statistical stylometric profiling into a single interactive visual dashboard.

### 2.3.2 How the Gap is Addressed by This Study

To address the limitations identified in existing literature, this study proposes a unified, lightweight, and offline academic integrity platform. The system bridges these technological and architectural gaps through four core interventions:

- **Dual-Engine Unified Architecture:** Instead of treating paraphrase detection and AI authorship as isolated tasks, the system integrates Sentence-BERT semantic vector embeddings (all-MiniLM-L6-v2) with statistical stylometric profiling (Perplexity and Burstiness) into a single analytical pipeline. This ensures simultaneous detection of semantic plagiarism and machine-generated text.
- **Semantic Vector Matching Over Lexical Overlap:** By replacing legacy N-gram string matching with dense 384-dimensional vector comparisons via Cosine Similarity

($\geq 0.75$), the system accurately flags context-preserving paraphrases, synonym substitutions, and passive-to-active voice shifts that bypass traditional tools.

- **Containerized, Privacy-Preserving Offline Execution:** The entire software framework is designed for local deployment using Docker or Python virtual environments. This eliminates reliance on third-party cloud APIs, guarantees 100% data privacy for student intellectual property, and removes recurring subscription costs.
- **Explainable Side-by-Side Visual Heatmap:** The system features an interactive React.js web interface that renders a side-by-side, color-coded sentence heatmap (Red = verbatim/high similarity, Yellow = paraphrased, Blue/Purple = AI-generated). This replaces opaque percentage scores with transparent, sentence-level visual evidence for educators.

## 2.5 Chapter Summary

This literature review establishes the theoretical, empirical, and practical foundations for the AI-Driven Academic Integrity System. The review synthesized three core theoretical models underpinning the platform: John Rupert Firth's Distributional Semantics and Vector Space Models for semantic sentence mapping via Sentence-BERT (all-MiniLM-L6-v2) and Cosine Similarity; Claude Shannon's Information Theory for assessing token predictability through document Perplexity; and Stylometric Variation Theory for evaluating sentence-length dynamics through Burstiness scoring.

Through comparative analysis of current solutions including Turnitin's lexical N-gram engine, GPTZero's standalone stylometrics, and SBERT offline semantic models the chapter highlighted critical vulnerabilities in existing paradigms, notably false-negative scores on paraphrased text, high commercial licensing costs, and data privacy risks associated with cloud uploads. Finally, the literature review identified the central research gap: the absence of a unified, 100% offline, privacy-preserving software platform that combines dense vector semantic matching with statistical stylometric AI profiling into a single interactive visual dashboard. The proposed system directly addresses this gap by delivering a containerized, hybrid dual-engine solution tailored for academic evaluation.

# CHAPTER 3: METHODOLOGY

## 3.1) Introduction

This chapter presents the methodology used to design, develop, test, and evaluate the proposed AI-Driven Academic Integrity System. Following the structured systems development framework adopted in the eWitness Vault methodology, the development process follows an Agile software design lifecycle divided into six distinct, iterative phases: requirements gathering, system design and architecture, dataset collection and preprocessing, system development, system testing and security validation, and final evaluation.

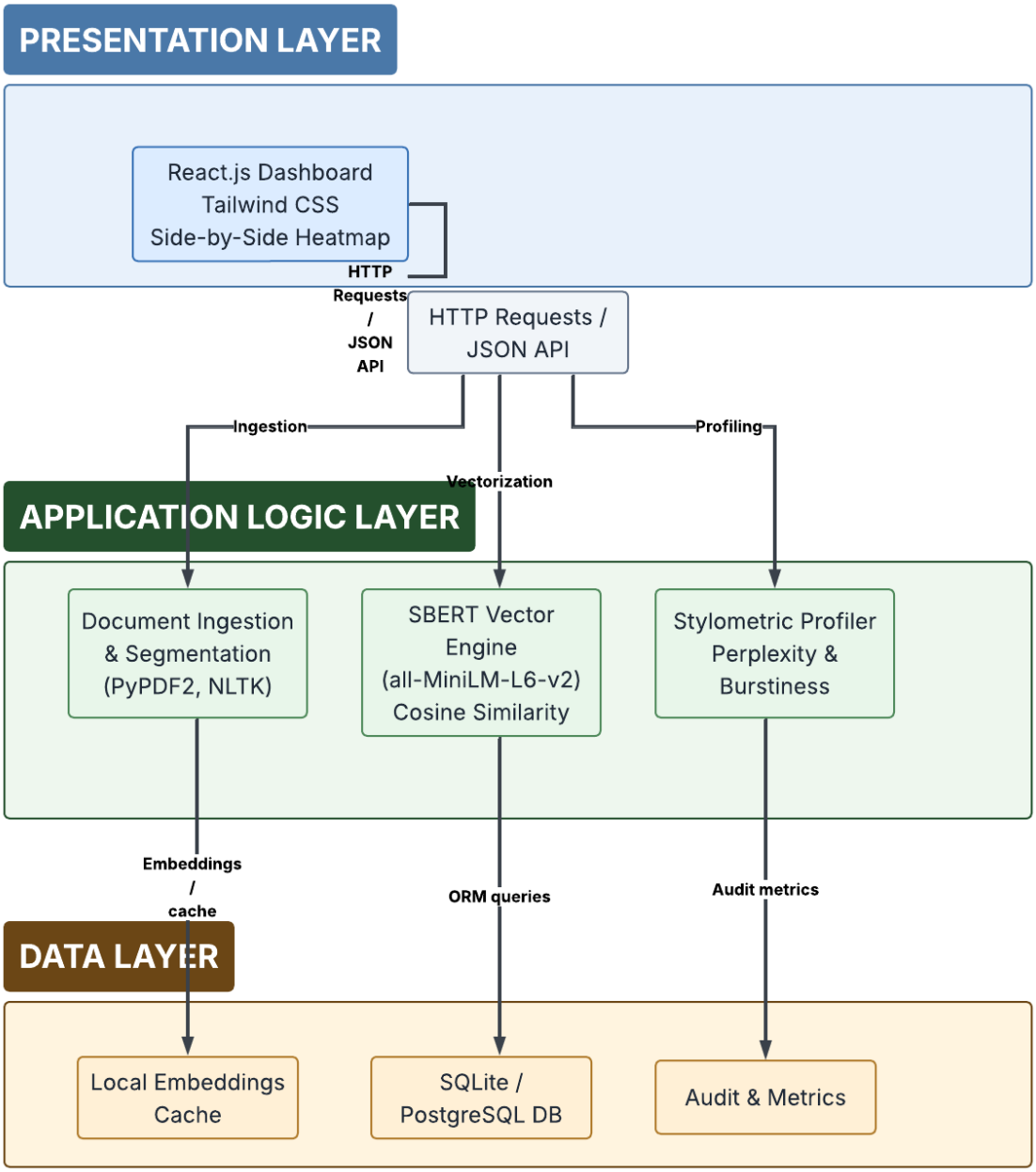
## 3.2) Requirement Gathering and Research Design

The requirement gathering and research design phase focuses on defining functional, technical, and domain-specific requirements for paraphrase detection and AI authorship profiling. Primary requirements will be gathered through three core activities:

- **Stakeholder Interviews:** Structured discussions with faculty members, academic integrity reviewers, and university IT administrators at UTAMU to identify current workflows, pain points in identifying AI-paraphrased assignments, and UI requirements for explainable feedback.
- **Document & Policy Review:** Analyzing institutional academic integrity policies and standard research submission guidelines to define evaluation metrics and baseline similarity thresholds.
- **Observation & Baseline Benchmarking:** Evaluating legacy string-matching and N-gram plagiarism checking software to document false-negative rates when processing paraphrased text and raw LLM output.

## 3.3) System Design

The system employs a modular, three-tiered **Model-View-Controller (MVC)** architecture designed for local, containerized execution to ensure 100% data privacy and offline operational capability:




- **Presentation Layer (Frontend):** Interactive React.js user interface styled with Tailwind CSS, featuring drag-and-drop file ingestion, real-time threshold controls, and a side-by-side color-coded sentence heatmap.
- **Application Logic Layer (Backend):** Python-based service layer consisting of:

    i. Document Ingestion & Parsing Module: Extracts raw text and parses PDFs/DOCX files into normalized, sentence-level units.

ii. SBERT Vector Matching Engine: Maps sentence vectors into a 384-dimensional space and executes Cosine Distance pairwise matching.

iii. Stylometric Profiling Engine: Evaluates token Perplexity and sentence-length Burstiness.

- **Data Layer:** Local relational database engine (SQLite/PostgreSQL) storing session metrics, execution logs, and embedded vector caches without sending data to external cloud APIs.

## 3.4) Dataset Collection and Preprocessing

To establish a rigorous baseline for testing system detection capabilities, a structured test dataset will be collected and preprocessed:

- **Dataset Composition:**
  - Human-Written Academic Corpus: Original student essays, research papers, and dissertations across computing and engineering disciplines.
  - AI-Paraphrased Corpus: Human text processed through automated rephrasing software (e.g., QuillBot) utilizing synonym substitution and passive-to-active voice shifts.
  - Fully AI-Generated Corpus: Text synthesized directly from Large Language Models (e.g., ChatGPT / GPT-4, Claude) across varied prompts.
- **Preprocessing Pipeline:**
  - Extraction of raw text from .pdf and .docx file formats using PyPDF2 and python-docx.
  - Text normalization (UTF-8 encoding standardization and cleanup).
  - Sentence-level segmentation using Python's Natural Language Toolkit (NLTK).

## 3.5) System Development

The software platform will be constructed using open-source, lightweight technologies optimized for edge deployment:

- **Backend Framework:** Python 3.11+, PyTorch, and Transformers/Sentence-Transformers library.
- **Machine Learning / NLP Models:** sentence-transformers/all-MiniLM-L6-v2 (384-dimensional dense vectors) and localized statistical language modeling scripts for Perplexity and Burstiness calculations.
- **Frontend Framework:** React.js integrated with Vite and Tailwind CSS.
- **Environment & Containerization:** Docker containerized virtual environment ensuring zero dependency on external third-party cloud APIs.
- **Version Control:** Git and GitHub repository management.

## 3.6) System Testing and Validation

System evaluation will be conducted through five distinct testing regimes to guarantee accuracy, performance, and operational security:

### 3.5.1) Functional Testing

Each independent component (document parser, sentence segmenter, vector generator, perplexity calculator, and frontend heatmap display) will undergo modular unit testing to verify computational correctness.

### 3.5.2) User Acceptance Testing (UAT)

A structured UAT pilot will be conducted with faculty reviewers and academic evaluators at UTAMU. Participants will interact with the React.js dashboard, test threshold sliders, and assess the clarity and explainability of the side-by-side color-coded heatmap visualizations.

### 3.5.3) Performance & Latency Testing

Execution benchmarking will evaluate file parsing throughput, SBERT vector encoding speeds, and total processing latency across varying document page lengths to confirm offline execution viability on local workstation hardware.

### 3.5.4) Security & Data Integrity Testing

Security assessments will verify local container isolation, confirming zero external network egress (preventing intellectual property leaks), file format upload validation

(blocking malicious executable uploads), and system robustness against adversarial paraphrasing prompt strategies.

### 3.5.5) Statistical Validation & Precision Metrics

The analytical accuracy of the SBERT semantic vector engine and stylometric profiler will be measured against labeled ground-truth evaluation datasets using standard metrics:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F}_1\text{-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Where:

- TP = True Positives (correctly flagged AI/paraphrased sentences)
- FP = False Positives (human text incorrectly flagged)
- FN = False Negatives (AI/paraphrased text missed by the system)

## 3.7) Summary

The methodology outlined in this chapter establishes a structured, end-to-end framework for developing and evaluating the proposed AI-Driven Academic Integrity System. By combining Agile software development iterations with rigorous empirical testing, the methodology bridges the operational gap between semantic paraphrase detection and stylometric AI content profiling within a unified application architecture.

Through the integration of local Python-based document processing, Sentence-BERT (all-MiniLM-L6-v2) vector embeddings, statistical Perplexity and Burstiness calculations, and an interactive React.js visual heatmap interface, this multi-phased approach guarantees that the resulting system delivers a lightweight, high-precision, and 100% privacy-preserving platform tailored to the academic integrity requirements of higher education institutions.

# References

Bird, S. L. (2023). Natural Language Processing with Python: Analyzing text with the Natural Language Toolkit (2nd ed.). O'Reilly Media.

Foundation, P. S. (2023). Python language reference (Version 3.11) [Computer software]. From https://www.python.org/

Jurafsky, D. &. (2023). Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition (3rd ed. draft ed.). Stanford University Press. . From https://web.stanford.edu/~jurafsky/slp3/

Mitchell, E. L. (2023). DetectGPT: Zero-shot machine-generated text detection using probability curvature. Proceedings of the 40th International Conference on Machine Learning (ICML), 202, 24950–24962. doi:https://proceedings.mlr.press/v202/mitchell23a.html

OpenAI. (2023). GPT-4 technical report (arXiv:2303.08774). arXiv. From https://doi.org/10.48550/arXiv.2303.08774

Reimers, N. &. (2023). Sentence-transformers: Fast sentence embeddings using Sentence-BERT models (all-MiniLM-L6-v2 documentation). Hugging Face Transformers Ecosystem. From https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

Source, M. O. (2023). React: A JavaScript library for building user interfaces (Version 18.2) [Computer software]. Meta Platforms. From https://react.dev/